

1. 走格子 (a.cpp)

题意

在 $n \times n$ 矩阵中从 $(1,1)$ 出发，先逐列右走，到本行末换到下一行第一列，走完最后一行最后一列后回到 $(1,1)$ 。求走 t 步后的位置。

思路

行走路线恰好把全部 n^2 个格子按行主序走成一圈，从 $(1,1)$ 开始，周期为 n^2 。

把位置映射为线性编号（从 0 开始）：

```
1 pos = (x - 1) * n + (y - 1)
```

走 t 步后编号为 $pos = t \bmod n^2$ ，还原：

```
1 x = pos / n + 1
2 y = pos % n + 1
```

- 若 $t \bmod n^2 == 0$ ，即停在起点 $(1,1)$ 。
- 数据范围 $n \leq 10$ ， $t \leq 500$ ，先取模再 $O(1)$ 计算；也可直接——模拟，最多 $n^2 \leq 100$ 步。

正确性

行走是"列优先、行次之、满周期回起点"的循环，位置号关于 n^2 取模即得余下步数的命中格，映射唯一，正确。

复杂度

- 时间复杂度：公式法 $O(1)$ ，模拟法 $O(n^2)$
- 空间复杂度： $O(1)$

关键点

- 周期取模：先 $t \% n^2$ ，把任意大步压缩到一圈内，避免 0 时刻特殊处理混乱。
- 编号与坐标的换算方向别写反（ $/$ 得行、 $\%$ 得列）。

2. 赛车 (b.cpp)

题意

$n \times m$ 网格，相邻格之间移动需适应力 $|a_i - a_j|$ 。任选起点，需"经过所有路标"完成比赛。求所需的最小适应力（即全程任一次移动所需边权的最大值）。

思路

把问题抽象成图：每个格子是一个点，相邻格子连边，边权为环境属性差。

- 适应力为 x 时，只能沿边权 $\leq x$ 的边走；
- "经过所有路标"等价于：**只保留边权 $\leq x$ 的边时，所有含路标的点都在同一个连通块内**（这样才能从块内任一点出发走遍全部路标）。

于是转化为**最小瓶颈连通**问题：不断加入边权最小的边，用**并查集**维护连通块及其含路标总数，当某个连通块的路标数达到全部路标数 `sum` 时，最后加入的那条边权就是答案。

- 若全图只有一个路标（`sum == 1`），无需移动，答案为 0。
- 把网格内所有横向、纵向相邻边收集起来，按边权升序排序（Kruskal）。

正确性

使所有路标连通的"最小最大边权"，正是按边权递增加边、直到路标同块时的当前边权；并查集合并过程天然按该顺序推进，故所得即最小值。

复杂度

- 时间复杂度： $O(nm \log(nm))$ （排序为主）
- 空间复杂度： $O(nm)$
- 规模：点 ≤ 250000 ，横纵边 $\leq$ 约 500000，可行。

关键点

- 把"经过所有路标"转成"路标点全连通"，再落到最小生成树的瓶颈思想。
- 并查集按秩/路径压缩，用一个 `siz` 记录块内路标数。
- 注意单独路标数为 1 时直接输出 0。

3. 背包问题 (c.cpp)

题意

n 个物品，选一个**价值和恰为 m** 的子集装包。到目的地前部分物品（含 0 个或全部）遗失。问背包里剩余价值和所有可能的取值，升序输出。

思路

每个物品有三种结局：**不选 / 选入且最终保留 / 选入但中途遗失**。

要求"装进背包的总值"恰好为 m ，同时关心"剩余（未遗失）的值"。设二维可行性 DP：

```
1 | dp[i][j] = 是否存在一种方案：装进背包的物品总值 = i，其中最终剩余（未遗失）的价值 = j
```

初始化 `dp[0][0] = true`。

对第 k 个价值为 x 的物品，倒序更新：

```
1 | 若选入但遗失: dp[i][j]   |= dp[i - x][j]           (i ≥ x)
2 | 若选入且保留: dp[i][j]   |= dp[i - x][j - x]       (i ≥ x 且 j ≥ x)
3 | 不选: dp[i][j] 保持继承
```

复杂度瓶颈是 x 与 dp 值域同为 m ，故采用两重倒序枚举（ i 从大到小、 j 从 i 到 0）防止同一物品被重复选取。

最终答案：所有满足 $dp[m][j] = \text{true}$ 的 j 升序输出。

正确性与暴力对照

- 暴力（测试点 1-3， $n \leq 8$ ）：直接枚举物品子集的幂次，验证选入总值 m 后剩余值， $O(2^n \cdot n)$ 。
- 正解：把"总值"与"剩余值"作为两维，三态转移完备覆盖所有可能，且倒序保证每物品只用一次，故正确。

复杂度

- 时间复杂度： $O(n \cdot m^2)$ （ $n, m \leq 500$ 时约 6×10^7 次布尔或运算，可行）
- 空间复杂度： $O(m^2)$

关键点

- 核心区别两个维度：**选入总值** vs **剩余值**，二者差即遗失部分。
- 倒序双重循环，保证每个物品只被考虑一次。
- $n, m, a_i \leq 500$ 均不大，二维表直接开 505×505 即可。

4. 相遇问题 (d.cpp)

题意

线段上有 n 个整数点，相邻点间有移动时间。两个物体**等速同时**从点 1 与点 n 出发相向而行，只能在**某个整数点**相遇。支持单点修改一段相邻边的移动时间，每次修改后求"可能的最短等待时间"最小值。

思路

设前缀和 $s[i]$ = 从点 1 到点 $i+1$ 的累计时间（ $s[0]=0$ ，总长 $s = s[n-1]$ ）。

若在点 p （坐标为第 p 个点）相遇：左物体走 $s[p-1]$ ，右物体从点 n 走 $s - s[p-1]$ 。二者等速同时出发，**行走距离（时间）之差就是先到者的等待时间**，故该点的代价为

$$1 \quad \text{cost}(p) = |s[p-1] - (s - s[p-1])| = |2 \cdot s[p-1] - s|$$

问题化为：在全部 n 个整数点中最小化 $|2 \cdot s[i] - s|$ 。

由于 s 关于位置单调不减， $|2 \cdot s[i] - s|$ 在 $s[i]$ 最接近 $s/2$ 时最小：

- 二分**找第一个满足 $2 \cdot s[\text{mid}] \geq s$ 的位置 res ，其代价 $2 \cdot s[\text{res}] - s$ ；
- 再与前一个位置 $\text{res}-1$ 的代价 $s - 2 \cdot s[\text{res}-1]$ 取较小者即为最优。

单点修改一条边会使其后所有前缀和同时变化，用**树状数组**维护前缀和：修改边 p 时对该边位置之后的累计做区间差值更新（单点加 delta ，前缀查询），每次询问 $O(\log n)$ 取前缀和。

正确性

代价定义源自等速相向移动的时间差，等价于 $|2s[i]-s|$ ； $s[i]$ 在取整前缀意义下关于位置单调，最小点必然落在跨过 $s/2$ 的相邻两位置之一，二分数值即可，正确。

复杂度

- 时间复杂度： $O((n+k)\log n)$
- 空间复杂度： $O(n)$
- 规模： $n, k \leq 2 \times 10^5$ ，可行；时间值最大 $2 \times 10^5 \times 10^6 = 2 \times 10^{11}$ ，需用 `long long`。

关键点

- 把"等待时间"等价于 $|左走距离 - 右走距离|$ ，去掉绝对值得最简形式。
- 前缀和因单点改整体漂移 → 用树状数组维护，避免每次重建。
- 二分找中位数附近的两个候选，比较取最小。
- 全程 `long long` 防溢出。

附录 · 数据规模一览

题目	程序	输入/输出	满数据规模	复杂度	主要算法
走格子	a.cpp	a.in/a.out	$n \leq 10, t \leq 500$	$O(1) / O(n^2)$	周期取模
赛车	b.cpp	b.in/b.out	$n, m \leq 500, a \leq 10^9$	$O(nm \log nm)$	Kruskal + 并查集
背包问题	c.cpp	c.in/c.out	$n, m, a \leq 500$	$O(nm^2)$	二维背包 DP
相遇问题	d.cpp	d.in/d.out	$n, k \leq 2 \times 10^5$	$O((n+k) \log n)$	树状数组 + 二分